

BÀI 1: LÀM QUEN VỚI PYTORCH

1. Mục Tiêu

- Sau khi hoàn thành bài thực hành, sinh viên có thể:
- Tạo, kiểm tra và thao tác với Tensor trong PyTorch.
- Thực hiện indexing, slicing, reshape và các phép toán ma trận trên Tensor.
- Sử dụng các hàm thống kê cơ bản trên Tensor.
- Hiểu cơ chế automatic differentiation (autograd) và vai trò của gradient.
- Sử dụng optimizer để giải bài toán tối ưu đơn giản bằng Gradient Descent.
- PyTorch cung cấp hai khả năng nền tảng: tính toán Tensor (tương tự NumPy, có thể tăng tốc bằng GPU) và automatic differentiation để huấn luyện mô hình.

2. Bài tập thực hành

Bài 1. Tạo Tensor cơ bản

Tạo một Tensor số nguyên kích thước 2×3 và hiển thị kết quả.

```
import torch

a = torch.tensor(
    [[1, 2, 3],
     [4, 5, 6]],
    dtype=torch.int32
)

print(a)
```

Yêu cầu: Cho biết kích thước, số chiều và kiểu dữ liệu của Tensor a.

Bài 2. Tạo các Tensor thường dùng

Thực hành tạo Tensor ngẫu nhiên, Tensor toàn số 0, toàn số 1 và ma trận đơn vị.

```
import torch

a = torch.rand(3, 4)
b = torch.randn(3, 4)
c = torch.zeros(2, 3)
d = torch.ones(2, 3)
e = torch.eye(4)

print(a)
print(b)
print(c)
print(d)
print(e)
```

Yêu cầu:

- Nhận xét sự khác nhau giữa torch.rand() và torch.randn().
- Tạo thêm một Tensor 5×5 chứa toàn giá trị 10.

Bài 3. Kiểm tra thông tin Tensor

Sử dụng shape, size(), ndim và dtype để kiểm tra các thuộc tính của Tensor.

```
import torch

a = torch.zeros(2, 3, 4)
```

```
print("Tensor:")
print(a)
print("Shape:", a.shape)
print("Size:", a.size())
print("Number of dimensions:", a.ndim)
print("Data type:", a.dtype)
```

Yêu cầu: Giải thích ý nghĩa của `torch.Size([2, 3, 4])`.

Bài 4. Slicing và Indexing

Truy xuất một phần dữ liệu từ Tensor nhiều chiều.

```
import torch

a = torch.randn(3, 4, 5)

print("Original tensor:")
print(a)

print("\nTensor a[1]:")
print(a[1])

print("\nTensor a[1:, 2:4]:")
print(a[1:, 2:4])
```

Yêu cầu:

- Cho biết shape của `a[1]` và `a[1:, 2:4]`.
- Thử lấy hai phần tử cuối cùng trên chiều thứ ba.

Bài 5. Reshape và Flatten

Thay đổi hình dạng Tensor mà không làm thay đổi số phần tử.

```
import torch

a = torch.randn(3, 4)

print("Original:")
print(a)

print("\nFlatten:")
print(a.ravel())

print("\nReshape to 3 x 2 x 2:")
print(a.reshape(3, 2, 2))
```

Yêu cầu:

- Kiểm tra số phần tử trước và sau khi reshape.
- Thử reshape Tensor thành kích thước 2×6 .

Bài 6. Phép toán và nhân ma trận

Thực hiện các phép toán phần tử và phép nhân ma trận bằng toán tử `@`.

```
import torch

a = torch.randn(2, 3)
b = torch.randn(2, 3)

print("a:")
print(a)
print("b:")
print(b)
```

```
print("\na + b:")
print(a + b)

print("\na / b:")
print(a / b)

print("\na^2:")
print(a ** 2)

print("\nMatrix multiplication:")
print(a @ b.T)
```

Yêu cầu:

- Phân biệt phép toán $a * b$ với $a @ b.T$.
- Cho biết shape của kết quả $a @ b.T$.

Bài 7. Các hàm thống kê trên Tensor

Tính trung bình, độ lệch chuẩn và tổng tích lũy theo từng cột.

```
import torch

a = torch.randn(3, 4)

print(a)
print("\nMean:")
print(torch.mean(a, dim=0))

print("\nStandard deviation:")
print(torch.std(a, dim=0))

print("\nCumulative sum:")
print(torch.cumsum(a, dim=0))
```

Yêu cầu:

- Thay $\text{dim}=0$ bằng $\text{dim}=1$ và nhận xét sự khác nhau.

Bài 8. Autograd cơ bản

Tạo biến có `requires_grad=True` và dùng `backward()` để tính đạo hàm tự động.

```
import torch

x = torch.tensor(
    3.6,
    requires_grad=True
)

y = x * x

y.backward()

print("x =", x)
print("y =", y)
print("x.grad =", x.grad)
```

Yêu cầu:

- Tính thủ công đạo hàm của $y = x^2$ tại $x = 3.6$ và so sánh với `x.grad`.
- Đổi hàm thành $y = 3x^2 + 2x + 1$ và kiểm tra gradient.

Bài 9. Gradient Descent tìm hệ số đa thức

Sinh dữ liệu từ đa thức $y = x^2 + 2x + 3$, sau đó dùng NAdam và autograd để tìm lại ba hệ số.

```
import numpy as np
import torch

polynomial = np.poly1d([1, 2, 3])
N = 20

X = np.random.randn(N, 1) * 5
Y = polynomial(X)

XX = np.hstack([
    X * X,
    X,
    np.ones_like(X)
])

w = torch.randn(3, 1, requires_grad=True)
x = torch.tensor(XX, dtype=torch.float32)
y = torch.tensor(Y, dtype=torch.float32)

optimizer = torch.optim.NAdam([w], lr=0.01)

print("Initial coefficients:")
print(w)

for epoch in range(1000):
    optimizer.zero_grad()
    y_pred = x @ w
    mse = torch.mean(torch.square(y - y_pred))
    mse.backward()
    optimizer.step()

print("Learned coefficients:")
print(w)
```

Yêu cầu:

- So sánh ba hệ số học được với [1, 2, 3].
- Giải thích vai trò của `optimizer.zero_grad()`, `mse.backward()` và `optimizer.step()`.

Bài 10. Giải hệ phương trình bằng Autograd

Tối thiểu hóa tổng sai số bình phương để tìm A, B, C, D thỏa bốn phương trình.

```
import random
import torch

A = torch.tensor(random.random(), requires_grad=True)
B = torch.tensor(random.random(), requires_grad=True)
C = torch.tensor(random.random(), requires_grad=True)
D = torch.tensor(random.random(), requires_grad=True)

EPOCHS = 2000
optimizer = torch.optim.NAdam([A, B, C, D], lr=0.01)

for _ in range(EPOCHS):
    y1 = A + B - 9
    y2 = C - D - 1
    y3 = A + C - 8
    y4 = B - D - 2

    sqerr = y1*y1 + y2*y2 + y3*y3 + y4*y4

    optimizer.zero_grad()
    sqerr.backward()
    optimizer.step()

print("A =", A)
print("B =", B)
```

```
print("C =", C)
print("D =", D)
```

Yêu cầu:

- Kiểm tra nghiệm thu được với bốn phương trình: $A+B=9$, $C-D=1$, $A+C=8$, $B-D=2$.
- Nhận xét vì sao bài toán có thể có nhiều nghiệm.

3. Bài tập áp dụng

Bài 11. Viết chương trình PyTorch để thực hiện các yêu cầu sau:

- Tạo Tensor X có 24 phần tử và reshape lần lượt thành 4×6 , 2×12 và $2 \times 3 \times 4$.
- Tạo hai Tensor A, B kích thước 3×4 ; tính $A+B$, $A*B$, A/B , trung bình theo cột và độ lệch chuẩn theo cột.
- Cho $y = 2x^2 + 5x + 3$. Dùng autograd tính dy/dx tại $x = 2$ và kiểm tra bằng phép tính thủ công.
- Thay đa thức trong Bài 9 bằng $y = 2x^2 - 3x + 5$; huấn luyện và cho biết các hệ số tìm được.

--- Hết ---